॥श्रीगणेशाय नमः॥

Local Search

Satisfaction problems:
- These are more concerned about reaching the goal state and worry less about the path taken.
- E.g. solving a Sudoku. It won't matter what cells we fill first.

Optimization problems:
- These generally have some sort of cost or objective function that is to be minimized.
- An optimization problem can be modeled as a satisfaction problem by adding some constraints (CSPs).

CSPs require one to satisfy a set of given constraints while constraint optimization problems require one to minimize the total cost of the unsatisfied constraints.

Local search

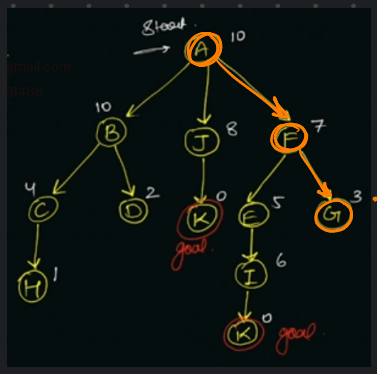
- These algos operate by searching from a start state to neighboring states without keeping track of the path.
- Optimal? No. They might not even explore a portion of the search space where an optimal solution lies.

Why use them?
- less memory requirements.
- Often we are okay with a sub-optimal but reasonable solution.

```
t = starting node
while (true):
    max_neighbor = t;
    for t' in neighbors(t):
        if f(t') > f(max_neighbor):
            max_neighbor = t'
    if f(max_neighbor) <= f(t):
        return t
    t = max_neighbor
```

↙ for maximizn

eg. consider the problem (minimization):



Here global minima is 0 but we get stuck at 3 (local minima).

↙ we'd stop here cause it won't go back to 7 (>3).

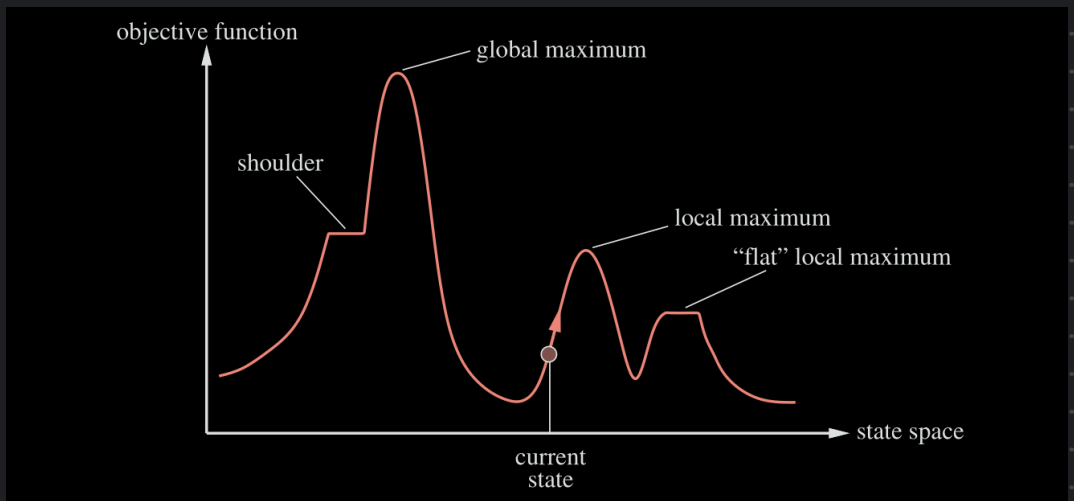
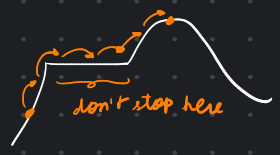


Figure 4.1 A one-dimensional state-space landscape in which elevation corresponds to the objective function. The aim is to find the global maximum.

Drawbacks
① local maxima
② plateau
③ shoulder
} termination condition
④ greedy nature: can't look beyond neighbors.

Escaping shoulders: sideways move
- keep moving sideways even if the objective function isn't increasing.



→ limit this to avoid ∞ loop

Tabu search:
- Keep the last k visited states in a fixed length queue (tabu list).
- Never make the step that is currently tabu'ed.
- A reasonable sized tabu list (100 or so) makes the search non-redundant.

Escaping shoulders: local optima enforced hill climbing
- Perform BFS from a local optima to find the next state with a better h function

Stochastic hill climbing

Random walk hill climbing
- At each step, do either greedy (with prob p move to next neighbor) or random (random neighbor with prob 1-p) step.

Hill climbing with random restarts
- For each restart, run until termination or for a fixed time.
- Run a fixed # of restarts or run indefinitely.

Hill climbing with both
- At each step, do greedy, random walk, or random restart.

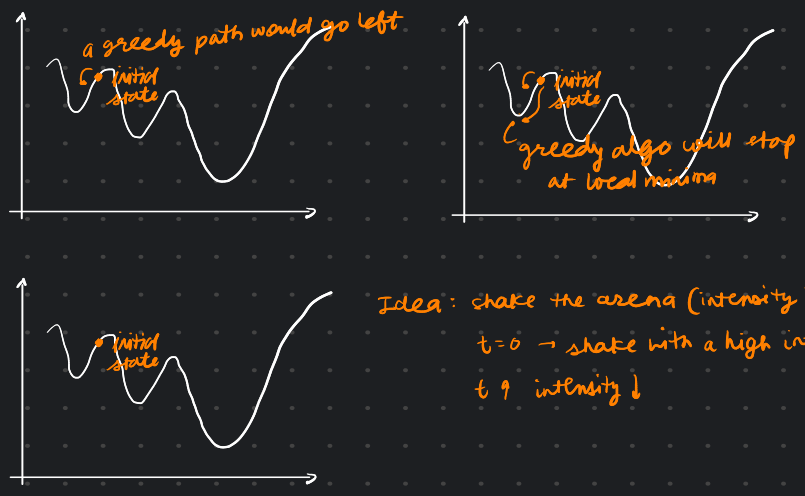
Simulated annealing

- like hill-climbing identify the quality of the local improvements
- instead of picking the best move, pick one randomly
- say the change in objective function is δ
- if δ is positive, then move to that state
- otherwise:
  - move to this state with probability proportional to δ
  - thus: worse moves (very large negative δ) are executed less often
- however, there is always a chance of escaping from local maxima
- over time, make it less likely to accept locally bad moves
- (Can also make the size of the move random as well, i.e., allow “large” steps in state space)

Simulated annealing

- heating a metal (a lot!) then cool down.
- done to increase ductility

say we are trying to find global minima



Idea: shake the arena (intensity is controllable)
t=0 → shake with a high intensity (hard shaking can skip a local minima)
t ↑ intensity ↓

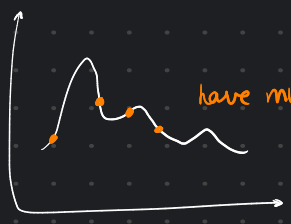
- Asymptotically complete.

```
SIMULATED_ANNEALING
current = initial_state
for t in (0, infinity):
    T = schedule(t)
    if T == 0:
        return current
    next = random neighbor of current
    ΔE = f(next) - f(current)
    if ΔE > 0:
        current = next
    else:
        current = next with probability e^(ΔE/T)
```

- Let $s = s_0$
- For $k = 0$ through k_{max} (exclusive):
  - $T \leftarrow$ temperature($1 - (k+1)/k_{max}$)
  - Pick a random neighbour, $s_{new} \leftarrow$ neighbour(s)
  - If $P(E(s), E(s_{new}), T) \geq$ random(0, 1):
    - $s \leftarrow s_{new}$
- Output: the final state s

- like hill-climbing identify the quality of the local improvements
- instead of picking the best move, pick one randomly
- say the change in objective function is δ
- if δ is positive, then move to that state
- otherwise:
  - move to this state with probability proportional to δ
  - thus: worse moves (very large negative δ) are executed less often
- however, there is always a chance of escaping from local maxima
- over time, make it less likely to accept locally bad moves
- (Can also make the size of the move random as well, i.e., allow "large" steps in state space)

Local beam search



It keeps track of k states rather than 1.



If no info is shared among peers ∴ equivalent of having k parallel random searches.
If info is being shared ∴ can't say equivalent to k || searches.

Stochastic beam search with k states